COLLEGE MANAGEMENT SYSTEM

Academic Project Report

Submitted in Partial Fulfillment of the Requirements for the Award of Bachelor of Computer Applications (BCA) / B.Sc. Information Technology

Detail	Information
Project Title	College Management System
Technology Stack	MongoDB, Express.js, React.js, Node.js (MERN Stack)
Project Level	Intermediate
Submitted By	[Student Name]
Roll Number	[Roll Number]
Department	Department of Computer Science & Applications
Institute	[Institute Name], [University Name]
Session	2025 – 2026
Guide	[Guide Name], [Designation]

CERTIFICATE

*This is to certify that the project report entitled **"College Management System"** is submitted by **[Student Name]**, Roll No. **[Roll Number]**, in partial fulfillment of the*

*requirements for the award of the degree of **Bachelor of Computer Applications**. This work has been carried out under my supervision and to my satisfaction.*

Project Guide: _____
Head of Department: _____
External Examiner: _____
Date: _____

DECLARATION

*I, **[Student Name]**, hereby declare that the project report entitled **"College Management System"** submitted to **[Institute Name]** for the partial fulfillment of the degree of **Bachelor of Computer Applications** is my original work. The work presented in this report has not been submitted elsewhere for any other degree or award. All information taken from other sources has been duly acknowledged.*

Student Signature: _____
Date: _____
Place: _____

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to all those who helped me complete this project successfully.

I am sincerely thankful to my project guide **[Guide Name]** for their constant support, thoughtful guidance, and valuable suggestions throughout the development of this project. Their mentorship made this challenging task achievable.

I am grateful to **[HOD Name]**, Head of the Department of Computer Science, for providing the necessary infrastructure and academic environment required for the project.

I also wish to acknowledge the contribution of my classmates and seniors who helped me during the testing phase.

Finally, I thank my family for being a constant source of encouragement throughout this academic journey.

[Student Name]
[Roll Number]
[Department]
[Institute Name]

CONTENT OF TABLE

S.NO.	TOPIC	PAGE NO.
1.	Title of the Project	1–6
2.	Introduction / Objective	7–10
3.	Gantt Chart	11
4.	Scope of the Project	12
5.	Methodology (SDLC, Functional Req., Non-Functional Req.)	13–16
6.	Software & Hardware Requirements	17–19
7.	Software Requirement Specification (SRS)	20–21
8.	System Design (DFD, E-R Diagram, Database Design)	22–26
9.	Structure & Database Specification	27–31
10.	Design: Output Screen of the Project	32–34
11.	Coding (React, Node.js, MongoDB)	35–65

S.NO.	TOPIC	PAGE NO.
12.	Testing & Debugging	66–70

SECTION 1 — TITLE OF THE PROJECT

1.1 Project Title

> ***College Management System*** *Built Using MERN Stack — MongoDB, Express.js, React.js, Node.js*

1.2 Abstract

The **College Management System** is a fully functional, full-stack web application designed to automate and digitize the administrative processes of a college or educational institution. Developed using the **MERN stack** (MongoDB, Express.js, React.js, Node.js), the system provides a centralized platform that manages students, teachers, staff, fees, payroll, attendance, library, exams, timetables, and reporting — all in one place.

The system enforces **role-based access control (RBAC)** with four distinct user roles: **Administrator, Teacher, Student, and Staff**. Each role receives a tailored dashboard and access to only the features relevant to their responsibilities. Authentication is secured using **JSON Web Tokens (JWT)** and password hashing with **bcryptjs**.

The backend API communicates with a **MongoDB Atlas** cloud database containing **20 collections** through the **Mongoose ODM**. The frontend, built with **React 18** and **Vite**, delivers a responsive and intuitive user interface accessible from any modern web browser.

Deployed on **Render.com**, this project replaces manual paper-based operations with a secure, scalable, and accessible digital solution that benefits all stakeholders in a college — from administrators to students.

1.3 Project Overview

The College Management System is a comprehensive web-based application that consolidates various college administrative tasks into a single, unified platform. The following is a brief overview of the major components of the system:

- **User Module:** Registration, login, and profile management for all four user types.
- **Student Module:** Enrollment, profile management, academic tracking, attendance, and grades.
- **Teacher Module:** Profile management, class assignment, attendance marking, and grade entry.
- **Staff Module:** Non-teaching staff records integrated with the payroll system.
- **Fee Module:** Fee type configuration, payment collection, and receipt generation.
- **Payroll Module:** Monthly salary calculation and disbursement for teachers and staff.
- **Attendance Module:** Daily attendance tracking with class-wise and student-wise reports.
- **Library Module:** Book catalogue management, book issuance, and return tracking.
- **Exam Module:** Exam session scheduling, marks entry, and result publication.
- **Timetable Module:** Weekly period-wise class schedule management.
- **Dashboard Module:** Real-time statistics and key metrics for administrators.
- **Reports Module:** Financial reports, attendance reports, and academic performance reports.
- **Notification Module:** System-wide alerts and notifications for all users.

The system operates on a **three-tier architecture**: a React-based presentation layer, an Express.js business logic layer, and a MongoDB data layer.

1.4 Problem Statement

Educational institutions today manage enormous volumes of administrative data — student records, fee transactions, attendance registers, examination results, salary sheets, library catalogs, and timetables. In most colleges, these tasks are handled through:

- Physical registers and paper files
- Disconnected Microsoft Excel spreadsheets
- Manual fee collection with handwritten receipts
- Printed timetables pinned to notice boards
- Manually calculated payroll sheets

This approach suffers from serious limitations:

1. **Data Redundancy** — The same student information is entered multiple times in different registers (admission, fee, attendance, library, exam), leading to duplication and inconsistency.
2. **Data Loss Risk** — Physical files can be misplaced, damaged by fire or water, or simply lost over time.
3. **Time Inefficiency** — Searching for a single student's complete record requires checking multiple files across multiple departments.
4. **No Real-Time Access** — There is no way to instantly view whether a student has paid fees, what their attendance percentage is, or what their exam grades are.
5. **Security Gaps** — Physical files have no access control — any staff member can potentially view or alter any record.
6. **Manual Calculation Errors** — Fee calculations, payroll computations, and grade totals are done manually, increasing the risk of arithmetic errors.
7. **No Centralized Reporting** — Generating a college-wide statistical report requires manually aggregating data from multiple departments.

These challenges demand a **modern, digital, and centralized solution** — the College Management System.

1.5 Aim of the Project

The primary aim of this project is to develop a **secure, scalable, and user-friendly College Management System** using the **MERN stack** that:

- Replaces all paper-based and spreadsheet-driven college processes with a centralized web application.
- Provides **role-specific dashboards** for Admins, Teachers, Students, and Staff.
- Secures all data access through **JWT authentication** and **role-based authorization**.
- Automates repetitive tasks such as fee receipt generation, payroll calculation, and attendance reports.
- Makes all college data accessible in **real time** from any internet-connected device.

1.6 Objectives

The specific objectives of the College Management System project are:

1. **To develop a full-stack web application** using MongoDB, Express.js, React.js, and Node.js (MERN stack) that covers all major college administrative operations.
2. **To implement secure user authentication** using JWT (JSON Web Tokens) with role-based access control (RBAC) supporting four roles: Admin, Teacher, Student, and Staff.
3. **To build a Student Management module** allowing administrators to add, edit, view, and delete student records, including personal details, academic information, and guardian contacts.
4. **To develop a Fee Management module** that records fee payments, generates unique receipts, displays payment history, and provides a financial summary.
5. **To create a Payroll Management module** that calculates and records monthly salary disbursements for both teaching and non-teaching staff in Indian Rupees (₹).
6. **To implement an Attendance Tracking module** that allows teachers to mark daily attendance and generate class-wise attendance reports.
7. **To build a Library Management module** supporting book catalogue management, book issuance, return tracking, and fine calculation.
8. **To develop an Exam and Grades module** for scheduling exam sessions, entering marks, calculating grades, and publishing results.
9. **To provide a Timetable Management module** for creating and viewing weekly class schedules.
10. **To deploy the application** on Render.com cloud hosting with MongoDB Atlas as the cloud database, ensuring 24/7 accessibility.
11. **To design a responsive UI** using React 18 and Tailwind CSS that works across desktops, tablets, and mobile devices.
12. **To generate administrative reports** including attendance summaries, financial statements, and academic performance overviews.

SECTION 2 — INTRODUCTION / OBJECTIVE

2.1 Background

The rapid digitization of educational institutions worldwide has made manual management systems obsolete. Colleges that continue to rely on paper-based records and disconnected spreadsheets face significant inefficiencies in their administrative operations. The demand for fast, accurate, and real-time access to college data has made web-based **College Management Systems (CMS)** an essential investment for modern educational institutions.

With the widespread adoption of the **MERN stack** in full-stack web development — combining the flexibility of JavaScript for both frontend and backend, the scalability of MongoDB's document database, and the efficiency of React's component-based UI — it has become possible to build highly capable and production-ready management systems at relatively low cost.

This project explores the design and development of a **web-based College Management System** using the MERN stack, with the goal of creating a complete and practical tool that can realistically be deployed in an actual college environment.

2.2 Need of the System

The need for a digital College Management System arises from the following organizational realities:

- **Volume of Data:** A typical college manages records for hundreds of students, dozens of teachers, administrative staff, thousands of fee transactions, and countless attendance entries per year.
- **Multi-Department Coordination:** The admissions office, accounts department, library, and examination cell all need access to the same student data but currently maintain separate files.
- **Regulatory Compliance:** Universities require colleges to submit accurate enrollment data, examination results, and financial reports — all of which must be precisely

compiled.

- **Student Expectations:** Today's students expect digital portals where they can view their grades, fees, timetables, and attendance online, without visiting the college office.
- **Security and Auditability:** Financial records and examination marks need to be protected from unauthorized access and tampering.

2.3 Existing System

In most colleges, the existing system consists of:

- **Paper-based student registers** maintained in the office, containing admission forms and student files.
- **Microsoft Excel sheets** for fee collection tracking, payroll sheets, and exam results.
- **Manual attendance registers** printed daily, filled by hand in each class, and stored in the department office.
- **Physical library cards** issued to students for book tracking.
- **Printed timetables** distributed at the start of each semester.
- **Notice boards** for communicating exam schedules and results to students.
- **Email or WhatsApp** for informal communication between staff and students.

2.4 Limitations of Existing System

#	Limitation	Impact
1	Data stored in physical files — risk of damage or loss	Permanent data loss if files are destroyed
2	No centralized database — each department maintains separate records	Data inconsistency and redundancy
3	Manual fee collection — no automatic receipt or payment history	Disputes over payments, manual errors

#	Limitation	Impact
4	Payroll calculated manually in Excel	Calculation errors in salary disbursement
5	No role-based access — any staff can view all files	Privacy and security violations
6	Attendance entered manually — prone to manipulation	Inaccurate attendance records
7	Library managed with physical cards — returns hard to track	Lost books and untracked fines
8	Results posted on notice boards — no digital access	Students must physically visit college
9	No real-time statistics available for management	Decisions made on outdated information
10	Reports require manual aggregation across departments	Preparation of reports takes days

---

2.5 Proposed System

The proposed **College Management System** is a web-based, full-stack application developed using the **MERN stack** that addresses every limitation listed above:

- **Centralized Database:** All college data is stored in a single **MongoDB Atlas** cloud database with 20 collections, accessible from any device with internet.
- **Role-Based Access Control (RBAC):** Four user roles — Admin, Teacher, Student, Staff — each with strict access permissions enforced via JWT middleware.
- **Automated Fee Receipts:** Payments are instantly recorded, and unique receipt numbers are generated automatically.
- **Payroll Processing:** Monthly salary calculation is automated with salary components (basic pay, allowances, deductions, net pay).
- **Digital Attendance:** Teachers mark attendance through the web interface; students can view real-time attendance percentage.

- **Library Management:** Book issuance and returns are tracked digitally with automatic fine calculation.
- **Online Results:** Grades are entered by the examination cell and published to students' dashboards.
- **Real-Time Reports:** Admin dashboard displays live statistics; reports can be generated and exported on demand.
- **Responsive UI:** React 18 with Tailwind CSS ensures the application works on desktops, tablets, and smartphones.
- **Cloud Deployment:** Hosted on **Render.com** (backend) and **MongoDB Atlas** (database) — available 24/7.

2.6 Advantages of Proposed System

1. **Single Source of Truth** — One database serves all departments; no duplication or conflict.
2. **24/7 Accessibility** — Accessible from any internet-connected device, anywhere.
3. **Improved Security** — JWT authentication, bcrypt password hashing, and RBAC ensure data is protected.
4. **Zero Paper Usage** — Completely paperless operations reduce administrative overhead.
5. **Instant Reports** — Financial, attendance, and academic reports generated within seconds.
6. **Audit Trail** — All transactions (payments, payroll, attendance) are timestamped and logged.
7. **Scalability** — MongoDB's document model and Node.js's non-blocking I/O scale easily with growing data.
8. **Cost Effective** — Built entirely with free and open-source tools; deploying on free tiers costs ₹0/month.
9. **Error Reduction** — Automated calculations replace manual arithmetic, eliminating calculation errors.
10. **Student Empowerment** — Students can independently view fees, grades, timetables, and attendance without visiting the office.

SECTION 3 — GANTT CHART

3.1 Project Timeline Overview

A **Gantt Chart** is a horizontal bar chart used in project management to visually represent the schedule of tasks over time. It shows the start date, duration, and end date of each project phase, making it easy to track project progress and ensure timely completion.

3.2 Project Phases & Tasks

Phase	Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8
1. Requirement	Project Study & Requirement Gathering	■	■						
	Feasibility Analysis		■	■					
	SRS Documentation			■	■				
2. Design	System Architecture Design				■				
	Database Schema Design				■	■			
	UI Wireframe & Mockup					■			

Phase	Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8
3. Coding	Backend API Development					▬	▬		
	Frontend Development (React)						▬	▬	
	Database Integration						▬	▬	
4. Testing	Unit Testing							▬	
	Integration Testing							▬	▬
	Bug Fixing								▬
5. Deployment	Deployment on Render.com								▬
	Final Report & Documentation								▬

Total Duration: 8 Weeks (2 Months)

3.3 Phase-wise Explanation

Phase 1 — Requirement Analysis (Week 1–3)

During this phase, the project requirements were gathered by studying the existing college management process. Stakeholder expectations were documented, feasibility was analyzed (technical, economic, operational), and the Software Requirements Specification (SRS) document was prepared.

Phase 2 — Design (Week 3–5)

System architecture (3-tier MERN model) was designed. Database schema of 20 MongoDB collections was planned out. DFD (Level 0 and Level 1), E-R Diagrams, and UI wireframes for all 24 pages were created.

Phase 3 — Coding (Week 5–7)

Backend development: 13 REST API controllers, 13 route files, 20 Mongoose models, JWT auth middleware, and `index.js` server setup. Frontend development: 24 React page components, 18 reusable components, Axios service layer, and Zustand auth store.

Phase 4 — Testing (Week 7–8)

Unit testing of individual API endpoints using Postman. Integration testing across modules. Bug fixing for identified issues. Final validation against functional requirements.

Phase 5 — Deployment (Week 8)

Backend deployed on **Render.com** as a Node.js Web Service. Frontend deployed as a **Static Site** on Render.com. MongoDB Atlas used as the cloud database. Environment variables configured and final end-to-end testing done.

SECTION 4 — SCOPE OF THE PROJECT

4.1 Scope

The **College Management System** covers the complete administrative workflow of a college institution. The scope of the project includes the following functional areas:

4.1.1 In Scope

- **Authentication & Authorization:** User registration, login, JWT-based session management, and role-based access for Admin, Teacher, Student, and Staff.
- **Student Management:** Full CRUD (Create, Read, Update, Delete) operations for student records including personal, academic, guardian, and address information.
- **Teacher Management:** CRUD for teacher records with department, qualification, salary, and subject assignment.
- **Staff Management:** CRUD for non-teaching staff with designation, department, and salary.
- **Class Management:** Creation and management of class groups with enrolled students and assigned teachers.
- **Fee Management:** Fee type configuration, student fee collection, receipt generation, and payment history.
- **Payroll Management:** Monthly salary processing for teaching and non-teaching staff.
- **Attendance Management:** Daily attendance marking and class-wise/student-wise report generation.
- **Library Management:** Book catalogue, issuance, return, and fine management.
- **Exam Management:** Exam session scheduling, marks entry, grade assignment, and result publication.
- **Timetable Management:** Weekly period-wise schedule creation and viewing for all classes.
- **Dashboard & Reports:** Real-time admin statistics, financial reports, and attendance reports.
- **Notifications:** System alerts and notifications for all user roles.

4.1.2 Out of Scope (Current Version)

- Online payment gateway integration (e.g., Razorpay, Paytm).
- Mobile application (Android/iOS).
- Biometric attendance integration.
- Email/SMS automated notifications.
- Video conferencing for online classes.

4.2 Future Enhancements

The following features are planned for future versions of the system:

1. **Online Fee Payment** — Integration with Razorpay or Paytm for digital fee payment by students.
2. **Mobile Application** — A React Native app for iOS and Android with push notifications.
3. **Email & SMS Alerts** — Automated reminders for fee dues, exam results, and attendance shortfalls.
4. **Bulk Import/Export** — CSV-based bulk import of student/teacher data and export of reports.
5. **Advanced Analytics** — AI-powered grade prediction, attendance trend analysis, and risk alerts.
6. **Parent Portal** — A dedicated login for parents to monitor their ward's academic progress.
7. **Video Conferencing** — Integration with Zoom or Google Meet API for online classes.
8. **Multi-Language Support** — Hindi and regional language interfaces for wider accessibility.
9. **Biometric Attendance** — Hardware fingerprint scanner integration for automatic attendance.
10. **Document Management** — Upload and store student documents (certificates, application forms).
11. **Alumni Portal** — A module for managing and connecting with college alumni.
12. **Hostel Management** — Room allocation, mess fees, and warden management module.

4.3 Target Users

User Role	Who They Are	System Usage
Administrator	College Principal, Registrar, or IT Admin	Full system management — students, teachers, fees, payroll, reports
Teacher	Lecturers, Professors, HODs	Mark attendance, enter grades, view timetable, view payslip
Student	Enrolled UG/PG students	View profile, fees, grades, attendance, timetable, library
Staff	Clerks, Lab Assistants, Peons, Security	View profile, view payslip, view timetable

SECTION 5 — METHODOLOGY

5.1 SDLC Model Used — Waterfall Model

The **Software Development Life Cycle (SDLC)** describes the structured process followed to design, develop, and test a software system. For this project, the **Waterfall Model** was chosen as the SDLC methodology.

What is the Waterfall Model?

The Waterfall Model is a **linear, sequential software development process** in which each phase must be completed before the next phase begins. The output of one phase serves as the input for the next phase. It was first described by Winston W. Royce in 1970 and is one of the most widely used models in academic software projects.

Phases of the Waterfall Model Applied to This Project

```
┌──────────────────────────────┐
│  1. REQUIREMENT ANALYSIS     │ ← Gather and document what the system must do
└──────────────┬───────────────┘
               ▼
┌──────────────────────────────┐
│  2. SYSTEM DESIGN            │ ← Plan architecture, DB schema, UI wireframes
└──────────────┬───────────────┘
               ▼
┌──────────────────────────────┐
│  3. IMPLEMENTATION (CODING)  │ ← Write backend API + frontend React code
└──────────────┬───────────────┘
               ▼
┌──────────────────────────────┐
│  4. TESTING                  │ ← Unit test, integration test, fix bugs
└──────────────┬───────────────┘
               ▼
┌──────────────────────────────┐
│  5. DEPLOYMENT               │ ← Deploy on Render.com + MongoDB Atlas
└──────────────┬
```

```
                ▼
┌──────────────────────────────┐
│  6. MAINTENANCE              │ ← Monitor, patch, and plan future
enhancements
└──────────────────────────────┘
```

Why Waterfall Was Chosen

- The project requirements were clearly defined from the start.
- Academic project timeline is fixed, making a sequential approach practical.
- Each phase's deliverables are well-understood (SRS → design docs → code → test report).
- Suitable for a team of 1–3 developers working on a bounded scope.

5.2 Functional Requirements

Functional requirements describe **what the system must do** — the specific behaviors and features it must support.

FR#	Module	Requirement
FR01	Auth	Users must be able to register with name, email, and password
FR02	Auth	Users must be able to log in with email and password and receive a JWT token
FR03	Auth	The system must support 4 roles: Admin, Teacher, Student, Staff
FR04	Auth	JWT tokens must expire after 24 hours; refresh tokens valid for 7 days
FR05	Student	Admin must be able to add a new student with full details (roll no., course, semester, section, DOB, guardian, address)
FR06	Student	Admin must be able to edit and delete student records

FR#	Module	Requirement
FR07	Student	Admin and Teacher must be able to search students by name or roll number
FR08	Teacher	Admin must be able to add, edit, and delete teacher records
FR09	Staff	Admin must be able to add, edit, and delete staff records
FR10	Class	Admin must be able to create class groups and assign students and teachers
FR11	Fee	Admin must be able to configure fee types (Tuition, Exam, Library, Hostel, etc.)
FR12	Fee	System must record fee payments and generate receipts with unique receipt numbers
FR13	Fee	Students must be able to view their payment history
FR14	Payroll	Admin must be able to process monthly payroll for all teaching and non-teaching staff
FR15	Payroll	System must calculate net salary = Basic + Allowances – Deductions
FR16	Payroll	Teachers and staff must be able to view their own payslip
FR17	Attendance	Teachers must be able to mark student attendance (Present/Absent/Late)
FR18	Attendance	System must generate attendance reports by class and date range
FR19	Library	Admin/Librarian must be able to add books with title, author, ISBN, category
FR20	Library	System must track book issuance and return; calculate overdue fines

FR#	Module	Requirement
FR21	Exam	Admin must be able to create exam sessions with subject, date, and time
FR22	Exam	Teachers must be able to enter marks per student; system auto-calculates grades
FR23	Exam	Admin can publish/unpublish results; students view published results
FR24	Timetable	Admin must be able to create a weekly timetable with periods, subjects, teachers
FR25	Dashboard	Admin dashboard must display total students, teachers, staff, classes, revenue, attendance %
FR26	Reports	System must generate financial reports of fee collection
FR27	Notifications	System must create and display notifications for all users

---

5.3 Non-Functional Requirements

Non-functional requirements describe **how the system performs** — quality attributes and constraints.

NFR#	Category	Requirement
NFR01	Performance	All API responses must be returned within 2 seconds under normal load
NFR02	Scalability	The system must support at least 100 concurrent users
NFR03	Security	All passwords must be encrypted using bcryptjs (salt rounds: 10)

NFR#	Category	Requirement
NFR04	Security	All API routes (except login/register) must require a valid JWT token
NFR05	Security	Role violations must be rejected with 403 Forbidden response
NFR06	Security	Rate limiting must restrict each IP to 100 requests per 15 minutes
NFR07	Availability	System must be available 24/7 with 99% uptime
NFR08	Usability	The UI must be intuitive; a new user must be able to navigate without training
NFR09	Compatibility	System must work on Chrome, Firefox, and Edge (version 90+)
NFR10	Responsiveness	UI must be responsive and usable on screen widths from 375px (mobile) to 1920px (desktop)
NFR11	Maintainability	Code must be modular, with controllers, routes, models in separate files
NFR12	Reliability	Database connections must use retry logic; server must not crash on unhandled errors
NFR13	Portability	Application must be deployable on any Node.js v16+ compatible server
NFR14	Compliance	Personal data (student records) should not be exposed without authentication

SECTION 6 — SOFTWARE & HARDWARE REQUIREMENTS

6.1 Software Requirements

6.1.1 Backend Software Requirements

Component	Software / Library	Version	Purpose
Runtime	Node.js	18.x LTS	JavaScript runtime for server-side code
Framework	Express.js	4.18+	HTTP server, REST API routing
Database	MongoDB Atlas	Cloud	NoSQL document database (cloud-hosted)
ODM	Mongoose	7.5+	MongoDB object modeling and schema validation
Authentication	jsonwebtoken	9.x	JWT token generation and verification
Password Hashing	bcryptjs	2.4+	Secure password encryption
Security	Helmet	7.x	HTTP security headers

Component	Software / Library	Version	Purpose
CORS	cors	2.8+	Cross-Origin Resource Sharing policy
Rate Limiting	express-rate-limit	7.x	API request throttling (100 req/15 min)
Validation	Joi	17.x	Request body schema validation
Environment	dotenv	16.x	Environment variable management
Language	JavaScript (CommonJS)	ES6+	Server-side programming language

6.1.2 Frontend Software Requirements

Component	Software / Library	Version	Purpose
UI Library	React	18.2+	Component-based UI development
Build Tool	Vite	4.x+	Fast development server and production builds
Routing	React Router DOM	6.x	Client-side page routing
State Management	Zustand	4.x	Global authentication state

Component	Software / Library	Version	Purpose
HTTP Client	Axios	1.x	HTTP requests to backend API
Styling	Tailwind CSS	3.3+	Utility-first CSS framework
Icons	Lucide React	Latest	SVG icon components
Notifications	react-hot-toast	2.x	Toast notification popups
Animation	Three.js	Latest	3D animated landing page hero
Charts	Recharts	2.x	Data visualization in dashboard
Language	JavaScript (JSX)	ES6+	Frontend programming language

6.1.3 Development & Deployment Tools

Tool	Version	Purpose
Visual Studio Code	Latest	Code editor
Git	2.x	Version control
GitHub	—	Remote repository hosting
Postman	Latest	API testing
MongoDB Compass	Latest	Database visualization tool
npm	9.x+	Package management

Tool	Version	Purpose
Render.com	—	Cloud deployment (backend + frontend)
MongoDB Atlas	Free Tier	Cloud database hosting

6.2 Hardware Requirements

6.2.1 Development Machine (Minimum)

Component	Specification
Processor	Intel Core i3 (8th Gen) or equivalent
RAM	8 GB (minimum), 16 GB (recommended)
Storage	20 GB free disk space
Display	1366 × 768 resolution or higher
Internet	Broadband connection (min. 10 Mbps)
Operating System	Windows 10 / 11, macOS 12+, or Ubuntu 20.04+

6.2.2 Server (Deployment — Render.com Free Tier)

Component	Specification
CPU	Shared CPU (0.1 vCPU — free tier)
RAM	512 MB RAM
Storage	Ephemeral (no persistent disk on free tier)
Network	Render CDN — global access
OS	Linux (Docker-based container)
Node.js	18.x (configured in render.yaml)

6.2.3 Cloud Database (MongoDB Atlas Free Tier)

Component	Specification
Storage	512 MB
RAM	Shared
Region	Mumbai (ap-south-1)
Backups	Daily snapshots
Access	Mongoose connection URI via environment variable

6.2.4 Client (End User) Requirements

Component	Minimum Requirement
Browser	Google Chrome 90+, Firefox 88+, or Microsoft Edge 90+
Internet	5 Mbps broadband
Screen	375px width (mobile) to 1920px (desktop)
JavaScript	Must be enabled in browser

6.3 Development Environment Setup

```
# 1. Clone the project repository
git clone https://github.com/[username]/college-management.git
cd college-management

# 2. Backend Setup
cd backend
npm install

# 3. Create backend environment file
cp .env.example .env
# Edit .env and fill in:
#   MONGODB_URI=mongodb+srv://...
```

```
#   JWT_SECRET=your-jwt-secret
#   JWT_REFRESH_SECRET=your-refresh-secret
#   PORT=5000
#   NODE_ENV=development

# 4. Start backend development server
npm run dev
# Backend runs on: http://localhost:5000

# 5. Frontend Setup (in a new terminal)
cd ../frontend
npm install

# 6. Start frontend development server
npm run dev
# Frontend runs on: http://localhost:5173
```

SECTION 7 — SOFTWARE REQUIREMENT SPECIFICATION (SRS)

7.1 Feasibility Analysis

A **Feasibility Study** is conducted before starting the development of any software project to determine whether the proposed system is technically, economically, and operationally viable. The following three feasibility analyses were performed for the College Management System:

7.1.1 Technical Feasibility

Technical feasibility assesses whether the required technology, tools, and expertise are available to build the proposed system.

Assessment:

Factor	Status
Programming Language — JavaScript (Node.js + React)	✅ Mature, widely supported, free
Database — MongoDB Atlas	✅ Cloud-hosted, free tier available
Framework — Express.js, React 18, Vite	✅ Stable, actively maintained open-source
Authentication — JWT + bcryptjs	✅ Industry standard, production-proven
Deployment — Render.com	✅ Free tier available, auto-deploy from Git
Developer Expertise — JavaScript, REST APIs, React	✅ Covered in BCA curriculum

Conclusion: The project is **technically feasible**. All required technologies are freely available, open-source, and well-documented. No proprietary or expensive tools are required.

7.1.2 Economic Feasibility

Economic feasibility evaluates the cost-benefit analysis of the proposed system.

Cost Analysis:

Item	Cost
Software Licenses	₹0 (all open-source)
Development Tools (VS Code, Postman)	₹0 (free)
MongoDB Atlas (Free Tier)	₹0/month
Render.com Deployment (Free Tier)	₹0/month
Domain Name (optional)	₹500–₹1,000/year
Developer Time (student project)	Academic investment
Total Monthly Running Cost	**₹0**

Benefits:

- Elimination of paper, printing, and physical file storage costs.
- Saves approximately 20–30 hours/month of administrative staff time.
- Reduces errors in fee calculations, payroll, and attendance records.
- Provides real-time access to data, eliminating delays in decision-making.

Conclusion: The project is **economically feasible**. The cost of development is minimal (free tools), and the long-term savings in time and resources easily justify the investment.

7.1.3 Operational Feasibility

Operational feasibility assesses whether the organization's users and staff can effectively use the proposed system.

Assessment:

Factor	Assessment
User Interface	Web browser — no installation needed, familiar to all users
Training Required	Minimal — intuitive UI with role-specific menus
Access Device	Any computer, tablet, or smartphone with internet
Resistance to Change	Low — system directly benefits all users
Admin Capability	Admin can manage everything without technical knowledge
Student Acceptance	High — students prefer digital portals to manual visits

Conclusion: The project is **operationally feasible**. The web-based interface requires no special hardware or software installation. Basic internet literacy is sufficient to operate the system.

7.2 Feasibility Study — Summary

Feasibility Type	Result	Justification
Technical Feasibility	✅ Feasible	All required technologies are free, open-source, and well-supported
Economic Feasibility	✅ Feasible	Zero licensing cost; free deployment on Render.com and MongoDB Atlas
Operational Feasibility	✅ Feasible	Browser-based, role-specific UI requires minimal training

> ***Overall Verdict:*** *The College Management System is* ***fully feasible*** *across all three dimensions and is recommended for development and deployment.*

SECTION 8 — SYSTEM DESIGN

8.1 Data Flow Diagram (DFD)

A **Data Flow Diagram (DFD)** is a graphical representation of how data flows through a system. It shows the sources and destinations of data, the processes that transform data, and the data stores used. DFDs are created at multiple levels — Level 0 (Context), Level 1 (overview), and Level 2 (detailed).

8.1.1 DFD — Level 0 (Context Diagram)

The Level 0 DFD shows the system as a single process (black box) and its interaction with external entities.

```
    Admin ──────────────────────────────────────────────►
   Teacher ─────────────────────────────────────────────►
┌──────────────────────────────────┐
   Student ──────────────────────── Login/Data Input ──► │
│
   Staff ────────────────────────────────────────────►    │  COLLEGE
MANAGEMENT SYSTEM   │
                                                          │
│
   Admin ◄──────────────────────────────────────────────  │     (Central
Process)        │
   Teacher ◄──────────────────────── Reports/Data ──────  │
│
   Student ◄────────────────────────────────────────────  │
│
   Staff ◄──────────────────────────────────────────────
└──────────────────────────────────┘
```

External Entities: - **Admin:** Manages all system data; receives reports and statistics. - **Teacher:** Marks attendance, enters grades; views timetable and payslip. - **Student:** Views grades, fees, attendance, timetable, and library. - **Staff:** Views payslip and timetable.

8.1.2 DFD — Level 1

The Level 1 DFD expands the central process into individual sub-processes:

```
                                ┌──────────────────────┐
User (any role) ──Login──▶ │  1.0 Authentication  │ ──JWT Token──▶ All
Modules
                                └──────────────────────┘
                                           │
                   ┌───────────────────────┼────────────────────────┐
                   ▼                       ▼                        ▼
  ┌──────────────────┐       ┌──────────────────┐
┌──────────────────────┐
  │ 2.0 Student Mgmt│       │ 3.0 Fee Module   │      │ 4.0 Payroll Module
│
  │ Add/Edit/Delete │       │ Pay/Receipt      │      │ Process/Payslip
│
  └──────────────────┘       └──────────────────┘
└──────────────────────┘
                   │                       │                        │
                   ▼                       ▼                        ▼
  ┌──────────────────┐       ┌──────────────────┐
┌──────────────────────┐
  │ 5.0 Attendance  │       │ 6.0 Library Mgmt │      │ 7.0 Exam & Grades  │
  │ Mark/Report     │       │ Issue/Return     │      │ Schedule/Marks/GPA │
  └──────────────────┘       └──────────────────┘
└──────────────────────┘
                   │                       │                        │
                   └───────────────────────┼────────────────────────┘
                                           ▼
                                ┌──────────────────────┐
                                │  MongoDB Atlas       │
                                │  (20 Collections)    │
                                └──────────────────────┘
```

8.2 E-R Diagram Explanation

An **Entity-Relationship (E-R) Diagram** represents the logical structure of a database. It shows the entities (tables/collections), their attributes (fields), and the relationships

between them.

Key Entities and Relationships

1. User — Student (One-to-One) Each Student record is linked to exactly one User account (for login). The `userId` field in the Student collection is a foreign key reference to the User collection.

2. User — Teacher (One-to-One) Each Teacher record is linked to exactly one User account. The `teacherProfileId` field in the User collection points to the Teacher record.

3. Student — Class (Many-to-One) Many students belong to one class. The `classId` field in the Student collection references the Class collection.

4. Student — Payment (One-to-Many) One student can have many payment records over time. The `studentId` field in the Payment collection references the Student.

5. Teacher — SalaryPayment (One-to-Many) One teacher can have many monthly salary payments. The `recipientId` field in SalaryPayment references Teacher or Staff.

6. Class — Timetable (One-to-Many) One class has many timetable slots (one per period per day). `classId` in Timetable references the Class.

7. Student — Attendance (One-to-Many) One student has many attendance records (one per school day). `studentId` in Attendance references Student.

8. Student — ExamResult (One-to-Many) One student has many exam results across subjects and sessions. `studentId` in ExamResult references Student.

9. Book — BookIssue (One-to-Many) One book can be issued to many students over time. `bookId` in BookIssue references Book.

8.3 Database Design

The database uses **MongoDB** (NoSQL document database) with **Mongoose** ODM. It follows a **document-based design** with references (ObjectId) between collections.

Design Principles Applied:

- **Normalization for core entities** (User, Student, Teacher): Separate collections with ObjectId references to avoid data duplication.
- **Embedding for nested data** (guardianInfo, address in Student): Embedded sub-documents used for closely related, non-shared data.
- **Indexes** on frequently queried fields (rollNo, email, userId, classId) for query performance.

SECTION 9 — STRUCTURE & DATABASE SPECIFICATION

9.1 Database Tables (Collections)

The College Management System uses **20 MongoDB collections**. Below are the detailed table structures:

9.1.1 Users Collection

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	MongoDB document ID
email	String	Unique, Required, Lowercase	User login email
password	String	Required	bcryptjs hashed password
fullName	String	Required	Full name of user
phone	String	Optional	Contact number
status	String	Enum: active/inactive/suspended	Account status
avatar	String	Optional	Profile picture URL
lastLogin	Date	Optional	Last login timestamp
roleAssignments	Array	—	Array of role objects
roleAssignments.roleId	ObjectId	FK → Role	Role assigned

Field	Data Type	Constraints	Description
roleAssignments.scopeId	ObjectId	FK → Class	Scope (for teachers)
roleAssignments.assignedAt	Date	Default: now	Assignment date
teacherProfileId	ObjectId	FK → Teacher	Teacher profile link
studentProfileId	ObjectId	FK → Student	Student profile link
createdAt	Date	Auto	Record creation time
updatedAt	Date	Auto	Last update time

9.1.2 Students Collection

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	MongoDB document ID
userId	ObjectId	FK → User, Required	Linked user account
parentId	ObjectId	FK → Parent	Guardian record
rollNo	String	Unique, Required	Student roll number
enrollmentYear	Number	Required	Year of admission
course	String	Required	BCA / B.Tech / B.Com etc.
semester	String	Required	Current semester (1–8)
section	String	Required	Section A / B / C
dateOfBirth	Date	Optional	Date of birth
guardianInfo.fatherName	String	Optional	Father’s name

Field	Data Type	Constraints	Description
guardianInfo.motherName	String	Optional	Mother's name
guardianInfo.fatherPhone	String	Optional	Father's phone
guardianInfo.motherPhone	String	Optional	Mother's phone
guardianInfo.emergencyContact	String	Optional	Emergency contact
address.street	String	Optional	Street address
address.city	String	Optional	City
address.state	String	Optional	State
address.pinCode	String	Optional	PIN code
address.country	String	Optional	Country
status	String	Enum: active/graduated/suspended	Enrollment status
createdAt	Date	Auto	Record creation time

Indexes: `rollNo` (unique), `course`, `userId`

9.1.3 Teachers Collection

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	MongoDB document ID
userId	ObjectId	FK → User, Required	Linked user account
employeeNo	String	Unique, Required	Teacher employee number
department	String	Required	Department name
qualification	String	Required	Academic qualification

Field	Data Type	Constraints	Description
specialization	String	Optional	Subject specialization
joiningDate	Date	Required	Date of joining
salary	Number	Required	Monthly salary (₹)
status	String	Enum: active/inactive	Employment status
createdAt	Date	Auto	Record creation time

9.1.4 Staff Collection

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	MongoDB document ID
userId	ObjectId	FK → User, Required	Linked user account
employeeNo	String	Unique, Required	Staff employee number
designation	String	Required	Job title
department	String	Required	Department
joiningDate	Date	Required	Date of joining
salary	Number	Required	Monthly salary (₹)
status	String	Enum: active/inactive	Employment status

9.1.5 Payments Collection

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	MongoDB document ID

Field	Data Type	Constraints	Description
studentId	ObjectId	FK → Student, Required	Student who paid
feeTypeId	ObjectId	FK → FeeType, Required	Type of fee paid
amount	Number	Required	Amount paid (₹)
paymentDate	Date	Default: now	Date of payment
method	String	Enum: Cash/UPI/Bank Transfer	Payment method
transactionRef	String	Optional	Transaction reference ID
receiptNo	String	Unique, Auto-generated	Receipt number
status	String	Enum: paid/pending/failed	Payment status

9.1.6 SalaryPayments Collection

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	MongoDB document ID
recipientId	ObjectId	FK → Teacher/Staff	Employee paid
recipientType	String	Enum: teacher/staff	Type of employee
month	String	Required	Pay month (e.g., Feb 2026)
basicSalary	Number	Required	Basic pay (₹)
allowances	Number	Default: 0	Total allowances (₹)
deductions	Number	Default: 0	Total deductions (₹)
netSalary	Number	Computed	Net salary = Basic + Allow – Ded
paidOn	Date	Default: now	Payment date
status	String	Enum: processed/pending	Payroll status

9.1.7 Attendance Collection

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	MongoDB document ID
studentId	ObjectId	FK → Student, Required	Student
classId	ObjectId	FK → Class, Required	Class
date	Date	Required	Attendance date
status	String	Enum: present/absent/late	Attendance status
markedBy	ObjectId	FK → Teacher	Teacher who marked
createdAt	Date	Auto	Timestamp

9.1.8 Books Collection (Library)

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	MongoDB document ID
title	String	Required	Book title
author	String	Required	Author name
isbn	String	Unique, Required	ISBN number
category	String	Required	Subject / genre
totalCopies	Number	Required	Total stock
availableCopies	Number	Required	Currently available
shelfLocation	String	Optional	Physical shelf location
addedDate	Date	Default: now	Date added to library

9.1.9 BookIssues Collection

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	MongoDB document ID
bookId	ObjectId	FK → Book, Required	Book issued
studentId	ObjectId	FK → Student, Required	Student borrower
issueDate	Date	Default: now	Issue date
dueDate	Date	Required	Return due date
returnDate	Date	Optional	Actual return date
fine	Number	Default: 0	Fine amount (₹/day)
status	String	Enum: issued/returned/overdue	Issue status

9.1.10 ExamResults Collection

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	MongoDB document ID
examSessionId	ObjectId	FK → ExamSession	Exam session
studentId	ObjectId	FK → Student, Required	Student
subjectId	ObjectId	FK → Subject	Subject
marksObtained	Number	Required	Marks scored
totalMarks	Number	Required	Maximum marks
grade	String	Auto-calculated	A+/A/B/C/D/F
isPublished	Boolean	Default: false	Visible to student?

9.1.11 Timetable Collection

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto	MongoDB document ID
classId	ObjectId	FK → Class, Required	Target class
day	String	Enum: Mon–Sat	Day of week
period	Number	Required	Period number (1–8)
subjectId	ObjectId	FK → Subject	Subject for this slot
teacherId	ObjectId	FK → Teacher	Assigned teacher
startTime	String	Required	e.g., “09:00”
endTime	String	Required	e.g., “10:00”

9.1.12 Full Collection List — 20 MongoDB Collections

#	Collection Name	Primary Key	Key Foreign Keys
1	User	_id	—
2	Role	_id	—
3	Student	_id	userId, parentId, classId
4	Teacher	_id	userId
5	Staff	_id	userId
6	Parent	_id	—
7	Class	_id	—
8	Subject	_id	classId

#	Collection Name	Primary Key	Key Foreign Keys
9	FeeType	_id	—
10	StudentFee	_id	studentId, feeTypeId
11	Payment	_id	studentId, feeTypeId
12	Receipt	_id	paymentId, studentId
13	Salary	_id	teacherId/staffId
14	SalaryComponent	_id	salaryId
15	SalaryPayment	_id	recipientId
16	ExamSession	_id	subjectId, classId
17	ExamResult	_id	examSessionId, studentId
18	Timetable	_id	classId, subjectId, teacherId
19	Project	_id	studentId, teacherId
20	Notification	_id	userId

SECTION 10 — DESIGN: OUTPUT SCREENS OF THE PROJECT

10.1 Login Page Design

The Login Page is the entry point of the College Management System. It is accessible at the `/login` route and provides:

Design Elements: - College Management System logo/title at the top center. - **Role Selector** — A pill-style tab row allowing the user to choose a role before logging in: Admin | Teacher | Student | Staff. - **Email Input Field** — Placeholder: "Enter your email". - **Password Input Field** — Password masked; includes a show/hide toggle. - **Login Button** — Full-width, primary color button labeled "Login". - **Forgot Password** link (future enhancement). - Animated background (gradient or star field).

Color Theme: Dark navy background with accent color based on role selected (blue for Admin, green for Teacher, purple for Student, orange for Staff).

Responsive: Single-column card layout, centered on screen. Works on mobile.

10.2 Admin Dashboard Design

The Admin Dashboard is the home screen after Admin login at `/dashboard`.

Design Elements: - **Top Header Bar:** College name, admin avatar, logout button, and theme toggle. - **Left Sidebar Navigation** with links to: Dashboard, Students, Teachers, Staff, Classes, Fees, Payroll, Attendance, Library, Exams, Timetable, Reports, Settings. - **Statistics Cards Row:** Four cards showing: - 🎓 Total Students (e.g., 240) - 👩‍🏫 Total Teachers (e.g., 18) - 🗂️ Total Staff (e.g., 12) - 🏫 Active Classes (e.g., 10) - **Second Row Cards:** - 💰 Revenue This Month (e.g., ₹1,24,500) - 📋 Pending Fees (e.g., ₹32,000) - 📊 Attendance Rate (e.g., 87%) - **Recent Activities Feed** — List of last 10 system actions. - **Quick Action Buttons:** Add Student, Add Teacher, Collect Fee, Process Payroll.

10.3 Forms Design

All data entry forms follow a consistent layout:

Student Registration Form (`/students/add`): - Section 1 — Basic Info: Full Name, Email, Roll No., Course, Semester, Section, DOB. - Section 2 — Guardian Info: Father Name, Mother Name, Father Phone, Mother Phone, Emergency Contact. - Section 3 — Address: Street, City, State, PIN Code, Country. - **Buttons:** Save Student (primary), Cancel (secondary).

Add Teacher Form (`/teachers/add`): - Employee Number, Department, Qualification, Specialization, Joining Date, Monthly Salary.

Fee Payment Form (`/fees`): - Student Search (by name/roll no.), Fee Type Selector, Amount, Payment Method (Cash/UPI/Bank Transfer), Transaction Reference. - After payment → Auto-generate receipt with unique Receipt Number.

Timetable Form (`/timetable`): - Class Selector, Day selector, Period number, Subject, Teacher, Start Time, End Time.

10.4 Reports Page Design

The Reports page at `/reports` provides:

- **Tab navigation:** Financial Report | Attendance Report | Academic Report.
- **Financial Report:** Date range picker, table showing student-wise fee collections, total revenue, pending dues.
- **Attendance Report:** Class selector, date range picker, attendance percentage per student in a sortable table.
- **Academic Report:** Exam session selector, subject-wise grade distribution in a bar chart (Recharts).
- **Export Option:** Download report as PDF/Excel (future enhancement).

10.5 Other Key Screens

Screen	Route	Description
Landing Page	`/`	Public page with hero animation (Three.js), features list, CTA buttons
Students List	`/students`	Paginated table with search/filter; Edit and Delete buttons per row
Student Details	`/students/:id`	Full student profile card with all details and fee/attendance summary
Teachers List	`/teachers`	Table of all teachers with department, qualification, salary
Library Page	`/library`	Book catalogue table; Issue Book form; Issued Books list
Timetable	`/timetable`	6×8 grid (Mon–Sat × 8 periods) for selected class
Payroll Page	`/payroll`	Salary sheet table; filter tabs (All/Faculty/Staff); Process button
Settings	`/settings`	Profile update, password change, notification preferences

SECTION 11 — CODING

11.1 Technology Stack Used

Layer	Technology
Frontend	React 18 + Vite + Tailwind CSS
Backend	Node.js + Express.js
Database	MongoDB Atlas (Mongoose ODM)
Authentication	JWT + bcryptjs
HTTP Client	Axios
State Management	Zustand

11.2 Database Connection Code

File: `backend/src/config/db.js`

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGODB_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
    console.log(`✅ MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`❌ DB Connection Error: ${error.message}`);
    process.exit(1);
  }
};
```

```
module.exports = connectDB;
```

11.3 Backend Server Entry Point

File: `backend/src/index.js`

```
 require('dotenv').config();
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const connectDB = require('./config/db');

const app = express();

// Connect to MongoDB
connectDB();

// Middleware
app.use(helmet());
app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }));
app.use(express.json());
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));

// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/students', require('./routes/student'));
app.use('/api/teachers', require('./routes/teacher'));
app.use('/api/staff', require('./routes/staff'));
app.use('/api/classes', require('./routes/class'));
app.use('/api/finance', require('./routes/finance'));
app.use('/api/payroll', require('./routes/payroll'));
app.use('/api/attendance', require('./routes/attendance'));
app.use('/api/library', require('./routes/library'));
app.use('/api/dashboard', require('./routes/dashboard'));
app.use('/api/reports', require('./routes/report'));
app.use('/api/notifications', require('./routes/notification'));

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`🚀 Server running on port ${PORT}`));
```

11.4 Authentication System

11.4.1 User Model

File: `backend/src/models/User.js`

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  email:    { type: String, unique: true, required: true, lowercase: true,
        trim: true },
  password: { type: String, required: true },
  fullName: { type: String, required: true },
  phone:    { type: String },
  status:   { type: String, enum: ['active','inactive','suspended'],
        default: 'active' },
  avatar:   { type: String },
  lastLogin: { type: Date },
  roleAssignments: [{
    roleId:     { type: mongoose.Schema.Types.ObjectId, ref: 'Role' },
    scopeId:    { type: mongoose.Schema.Types.ObjectId, ref: 'Class' },
    assignedAt: { type: Date, default: Date.now },
    assignedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  }],
  teacherProfileId: { type: mongoose.Schema.Types.ObjectId, ref: 'Teacher'
        },
  studentProfileId: { type: mongoose.Schema.Types.ObjectId, ref: 'Student'
        },
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

11.4.2 Auth Controller

File: `backend/src/controllers/auth.js`

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');
```

```
// Register
exports.register = async (req, res) => {
  try {
    const { email, password, fullName, phone } = req.body;
    const existing = await User.findOne({ email });
    if (existing) return res.status(409).json({ message: 'Email already
        registered' });

    const hashed = await bcrypt.hash(password, 10);
    const user = await User.create({ email, password: hashed, fullName,
        phone });
    res.status(201).json({ message: 'User registered successfully',
        userId: user._id });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// Login
exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email
        }).populate('roleAssignments.roleId');
    if (!user) return res.status(401).json({ message: 'Invalid
        credentials' });

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(401).json({ message: 'Invalid
        credentials' });

    const token = jwt.sign({ userId: user._id }, process.env.JWT_SECRET, {
        expiresIn: '24h' });
    const refreshToken = jwt.sign({ userId: user._id },
        process.env.JWT_REFRESH_SECRET, { expiresIn: '7d' });

    user.lastLogin = new Date();
    await user.save();

    res.json({
      token,
      refreshToken,
      user: {
        id: user._id,
        email: user.email,
        fullName: user.fullName,
        role: user.roleAssignments[0]?.roleId?.name || 'student',
      }
    });
```

```
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// Get current user
exports.getMe = async (req, res) => {
  const user = await User.findById(req.user._id).select('-password');
  res.json(user);
};
```

11.4.3 Auth Middleware

File: `backend/src/middleware/auth.js`

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

exports.protect = async (req, res, next) => {
  try {
    const token = req.headers.authorization?.split(' ')[1];
    if (!token) return res.status(401).json({ message: 'Not authorized, no
        token' });

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.userId)
      .select('-password')
      .populate('roleAssignments.roleId');
    next();
  } catch (err) {
    res.status(401).json({ message: 'Token invalid or expired' });
  }
};

exports.authorize = (...roles) => (req, res, next) => {
  const userRole = req.user?.roleAssignments?.[0]?.roleId?.name;
  if (!roles.includes(userRole)) {
    return res.status(403).json({ message: `Role '${userRole}' is not
        authorized` });
  }
  next();
};
```

11.5 CRUD Operations — Student Module

11.5.1 Student Model

File: `backend/src/models/Student.js`

```
const mongoose = require('mongoose');

const studentSchema = new mongoose.Schema({
  userId:        { type: mongoose.Schema.Types.ObjectId, ref: 'User',
        required: true },
  parentId:      { type: mongoose.Schema.Types.ObjectId, ref: 'Parent' },
  rollNo:        { type: String, unique: true, required: true },
  enrollmentYear:{ type: Number, required: true },
  course:        { type: String, required: true },
  semester:      { type: String, required: true },
  section:       { type: String, required: true },
  dateOfBirth:   { type: Date },
  guardianInfo: {
    fatherName: String, motherName: String,
    fatherPhone: String, motherPhone: String,
    emergencyContact: String,
  },
  address: {
    street: String, city: String,
    state: String, pinCode: String, country: String,
  },
  status: { type: String, enum:
        ['active','inactive','graduated','suspended'], default: 'active'
        },
}, { timestamps: true });

studentSchema.index({ rollNo: 1 });
studentSchema.index({ course: 1 });
studentSchema.index({ userId: 1 });

module.exports = mongoose.model('Student', studentSchema);
```

11.5.2 Student Controller (CRUD)

File: `backend/src/controllers/student.js`

```
const Student = require('../models/Student');
const User = require('../models/User');

// GET all students (paginated)
exports.getAllStudents = async (req, res) => {
  try {
    const page  = parseInt(req.query.page)  || 1;
    const limit = parseInt(req.query.limit) || 25;
    const search = req.query.search || '';

    const filter = search
      ? { $or: [{ rollNo: new RegExp(search, 'i') }, { course: new
        RegExp(search, 'i') }] }
      : {};

    const total = await Student.countDocuments(filter);
    const students = await Student.find(filter)
      .populate('userId', 'fullName email phone status')
      .skip((page - 1) * limit)
      .limit(limit)
      .sort({ createdAt: -1 });

    res.json({ students, total, page, pages: Math.ceil(total / limit) });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// GET student by ID
exports.getStudentById = async (req, res) => {
  try {
    const student = await Student.findById(req.params.id)
      .populate('userId', 'fullName email phone status')
      .populate('parentId');
    if (!student) return res.status(404).json({ message: 'Student not
        found' });
    res.json(student);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// POST create student
exports.createStudent = async (req, res) => {
  try {
```

```
    const { rollNo, enrollmentYear, course, semester, section,
            dateOfBirth, guardianInfo, address, userId } = req.body;

    const existing = await Student.findOne({ rollNo });
    if (existing) return res.status(409).json({ message: 'Roll number
        already exists' });

    const student = await Student.create({
      userId, rollNo, enrollmentYear, course, semester,
      section, dateOfBirth, guardianInfo, address,
    });
    res.status(201).json({ message: 'Student created', student });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// PUT update student
exports.updateStudent = async (req, res) => {
  try {
    const student = await Student.findByIdAndUpdate(
      req.params.id, req.body, { new: true, runValidators: true }
    );
    if (!student) return res.status(404).json({ message: 'Student not
        found' });
    res.json({ message: 'Student updated', student });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// DELETE student
exports.deleteStudent = async (req, res) => {
  try {
    const student = await Student.findByIdAndDelete(req.params.id);
    if (!student) return res.status(404).json({ message: 'Student not
        found' });
    res.json({ message: 'Student deleted successfully' });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};
```

11.5.3 Student Routes

File: `backend/src/routes/student.js`

```
const router = require('express').Router();
const { protect, authorize } = require('../middleware/auth');
const ctrl = require('../controllers/student');

router.get('/',    protect, ctrl.getAllStudents);
router.get('/:id', protect, ctrl.getStudentById);
router.post('/',   protect, authorize('admin'), ctrl.createStudent);
router.put('/:id', protect, authorize('admin'), ctrl.updateStudent);
router.delete('/:id', protect, authorize('admin'), ctrl.deleteStudent);

module.exports = router;
```

11.6 Fee Payment Processing

File: `backend/src/controllers/finance.js`

```
const Payment = require('../models/Payment');
const StudentFee = require('../models/StudentFee');

// Process fee payment
exports.processPayment = async (req, res) => {
  try {
    const { studentId, feeTypeId, amount, method, transactionRef } =
        req.body;

    const receiptNo = `RCP-${Date.now()}`;
    const payment = await Payment.create({
      studentId, feeTypeId, amount, method,
      transactionRef, receiptNo, status: 'paid',
    });

    // Update student fee status
    await StudentFee.findOneAndUpdate(
      { studentId, feeTypeId },
      { $set: { status: 'paid', paidDate: new Date() } }
    );

    res.status(201).json({
      message: 'Payment recorded successfully',
      receiptNo,
```

```
      payment,
    });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// Get payment history
exports.getPaymentHistory = async (req, res) => {
  try {
    const payments = await Payment.find({ studentId: req.params.id })
      .populate('feeTypeId', 'name')
      .sort({ paymentDate: -1 });
    res.json(payments);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};
```

11.7 Frontend Code — React Components

11.7.1 Main App (Routing)

File: `frontend/src/App.jsx`

```
 import { BrowserRouter, Routes, Route } from 'react-router-dom';
import ProtectedRoute from './components/ProtectedRoute';
import LandingPage from './pages/LandingPage';
import LoginPage from './pages/LoginPage';
import Dashboard from './pages/Dashboard';
import StudentsPage from './pages/StudentsPage';
import TeachersPage from './pages/TeachersPage';
import FeesPage from './pages/FeesPage';
import AttendancePage from './pages/AttendancePage';
import LibraryPage from './pages/LibraryPage';
import PayrollPage from './pages/PayrollPage';

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
```

```
        <Route path="/"      element={<LandingPage />} />
        <Route path="/login" element={<LoginPage />} />
        <Route element={<ProtectedRoute />}>
          <Route path="/dashboard"  element={<Dashboard />} />
          <Route path="/students"   element={<StudentsPage />} />
          <Route path="/teachers"   element={<TeachersPage />} />
          <Route path="/fees"       element={<FeesPage />} />
          <Route path="/attendance" element={<AttendancePage />} />
          <Route path="/library"    element={<LibraryPage />} />
          <Route path="/payroll"    element={<PayrollPage />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

11.7.2 Auth Store (Zustand)

File: `frontend/src/store/auth.js`

```
 import { create } from 'zustand';
import { persist } from 'zustand/middleware';
import api from '../services/api';

export const useAuthStore = create(persist((set) => ({
  user: null,
  token: null,
  isAuthenticated: false,

  login: async (email, password) => {
    const res = await api.post('/auth/login', { email, password });
    set({ user: res.data.user, token: res.data.token, isAuthenticated:
        true });
    return res.data;
  },

  logout: () => set({ user: null, token: null, isAuthenticated: false }),
}), { name: 'auth-storage' }));
```

11.7.3 Axios API Service

File: `frontend/src/services/api.js`

```
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api',
  timeout: 10000,
});

// Attach JWT token to every request
api.interceptors.request.use((config) => {
  const auth = JSON.parse(localStorage.getItem('auth-storage') || '{}');
  const token = auth?.state?.token;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Handle 401 globally
api.interceptors.response.use(
  (res) => res,
  (error) => {
    if (error.response?.status === 401) window.location.href = '/login';
    return Promise.reject(error);
  }
);

export default api;
```

11.7.4 Login Page Component

File: `frontend/src/pages/LoginPage.jsx`

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuthStore } from '../store/auth';
import toast from 'react-hot-toast';

const ROLES = ['admin', 'teacher', 'student', 'staff'];
```

```
export default function LoginPage() {
  const [role, setRole]         = useState('admin');
  const [email, setEmail]       = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading]   = useState(false);
  const login    = useAuthStore((s) => s.login);
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    try {
      await login(email, password);
      toast.success('Login successful!');
      navigate('/dashboard');
    } catch (err) {
      toast.error(err.response?.data?.message || 'Login failed');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen bg-gray-900 flex items-center justify-
        center">
      <div className="bg-gray-800 rounded-2xl p-8 w-full max-w-md shadow-
        2xl">
        <h1 className="text-2xl font-bold text-white text-center mb-6">
          College Management System
        </h1>

        {/* Role Selector */}
        <div className="flex gap-2 mb-6">
          {ROLES.map((r) => (
            <button key={r}
              onClick={() => setRole(r)}
              className={`flex-1 py-2 rounded-lg text-sm font-medium
        capitalize transition
                ${role === r ? 'bg-blue-600 text-white' : 'bg-gray-700
        text-gray-300'}`}
            >
              {r}
            </button>
          ))}
        </div>

        <form onSubmit={handleSubmit} className="space-y-4">
          <input type="email" placeholder="Email address"
```

```
            value={email} onChange={(e) => setEmail(e.target.value)}
            required
            className="w-full px-4 py-3 rounded-lg bg-gray-700 text-white
        border border-gray-600 focus:outline-none focus:border-blue-500"
          />
          <input type="password" placeholder="Password"
            value={password} onChange={(e) => setPassword(e.target.value)}
            required
            className="w-full px-4 py-3 rounded-lg bg-gray-700 text-white
        border border-gray-600 focus:outline-none focus:border-blue-500"
          />
          <button type="submit" disabled={loading}
            className="w-full py-3 bg-blue-600 text-white rounded-lg font-
        semibold hover:bg-blue-700 disabled:opacity-50 transition"
          >
            {loading ? 'Logging in...' : 'Login'}
          </button>
        </form>
      </div>
    </div>
  );
}
```

11.7.5 Protected Route Component

File: `frontend/src/components/ProtectedRoute.jsx`

```
import { Navigate, Outlet } from 'react-router-dom';
import { useAuthStore } from '../store/auth';
import Layout from './Layout';

export default function ProtectedRoute() {
  const isAuthenticated = useAuthStore((s) => s.isAuthenticated);
  if (!isAuthenticated) return <Navigate to="/login" replace />;
  return <Layout><Outlet /></Layout>;
}
```

11.7.6 Dashboard Component

File: `frontend/src/pages/Dashboard.jsx` **(excerpt)**

```
import { useEffect, useState } from 'react';
import api from '../services/api';

export default function Dashboard() {
  const [stats, setStats] = useState(null);

  useEffect(() => {
    api.get('/dashboard/stats').then((res) => setStats(res.data));
  }, []);

  if (!stats) return <p className="text-white">Loading...</p>;

  const cards = [
    { label: 'Total Students', value: stats.totalStudents, icon: '🎓',
        color: 'blue' },
    { label: 'Total Teachers', value: stats.totalTeachers, icon: '👩‍🏫',
        color: 'green' },
    { label: 'Total Staff',    value: stats.totalStaff,    icon: '🗂️',
        color: 'yellow' },
    { label: 'Active Classes', value: stats.totalClasses,  icon: '🏫',
        color: 'purple' },
  ];

  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold text-white mb-6">Admin
        Dashboard</h1>
      <div className="grid grid-cols-4 gap-4 mb-6">
        {cards.map((c) => (
          <div key={c.label} className={`bg-gray-800 rounded-xl p-5 border
        border-${c.color}-500/30`}>
            <div className="text-3xl mb-2">{c.icon}</div>
            <div className="text-3xl font-bold text-white">{c.value}</div>
            <div className="text-gray-400 text-sm mt-1">{c.label}</div>
          </div>
        ))}
      </div>
    </div>
  );
}
```

11.7.7 Dashboard Controller (Backend)

File: `backend/src/controllers/dashboard.js`

```
 const Student = require('../models/Student');
const Teacher = require('../models/Teacher');
const Staff   = require('../models/Staff');
const Class   = require('../models/Class');
const Payment = require('../models/Payment');

exports.getStats = async (req, res) => {
  try {
    const [totalStudents, totalTeachers, totalStaff, totalClasses] = await
        Promise.all([
      Student.countDocuments({ status: 'active' }),
      Teacher.countDocuments({ status: 'active' }),
      Staff.countDocuments({ status: 'active' }),
      Class.countDocuments(),
    ]);

    // Revenue this month
    const startOfMonth = new Date(new Date().getFullYear(), new
        Date().getMonth(), 1);
    const revenueResult = await Payment.aggregate([
      { $match: { paymentDate: { $gte: startOfMonth }, status: 'paid' } },
      { $group: { _id: null, total: { $sum: '$amount' } } },
    ]);
    const revenue = revenueResult[0]?.total || 0;

    res.json({ totalStudents, totalTeachers, totalStaff, totalClasses,
        revenue });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};
```

SECTION 12 — TESTING & DEBUGGING

12.1 Testing Methods

Testing is the process of evaluating a software system to verify that it satisfies the specified requirements and to identify defects. The following testing methods were applied:

12.1.1 Unit Testing

Individual functions and API endpoints were tested in isolation using **Postman**. Each controller function (login, createStudent, payFee, etc.) was tested with valid input, invalid input, and boundary cases.

12.1.2 Integration Testing

Multiple modules were tested together to ensure they work correctly as a combined system. For example: - Student creation + Fee assignment + Payment processing was tested end-to-end. - Teacher login + Attendance marking + Report generation was tested.

12.1.3 User Acceptance Testing (UAT)

The fully integrated system was tested by simulating the actual workflow of each user role: - Admin logged in and performed student management, fee collection, and payroll processing. - Teacher logged in, marked attendance, entered grades, and viewed payslip. - Student logged in, viewed attendance, grades, fees paid, and timetable.

12.1.4 Security Testing

- Attempted access to protected routes without a token — verified 401 Unauthorized response.
- Tested Student role accessing Admin-only routes — verified 403 Forbidden response.
- Verified passwords are stored as bcryptjs hashes in MongoDB (not plain text).
- Tested rate limiter with 105 rapid requests — verified 429 Too Many Requests after 100.

12.2 Test Cases Table

Module: Authentication

TC#	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC01	Login with valid Admin credentials	admin@college.com / admin123	JWT token returned, role = admin	JWT token returned	✅ Pass
TC02	Login with wrong password	admin@college.com / wrong	401 “Invalid credentials”	401 returned	✅ Pass
TC03	Login with unregistered email	fake@test.com / pass	401 “Invalid credentials”	401 returned	✅ Pass
TC04	Access protected route without token	No Authorization header	401 “Not authorized, no token”	401 returned	✅ Pass
TC05	Access protected route with valid token	Bearer [valid token]	200 OK with data	200 OK returned	✅ Pass
TC06	Access protected route with expired token	Bearer [expired token]	401 “Token invalid or expired”	401 returned	✅ Pass

---

Module: Student Management

TC#	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC07	Add student with all valid fields	Complete student data	201 Student created	201 returned	✅ Pass
TC08	Add student with duplicate roll number	Existing rollNo value	409 “Roll number already exists”	409 returned	✅ Pass

TC#	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC09	Add student without required field (course)	Missing course field	400 Validation error	400 returned	✅ Pass
TC10	Get all students	GET /api/students	Paginated student list	List returned	✅ Pass
TC11	Get student by valid ID	Valid MongoDB ObjectId	Student detail object	Student data returned	✅ Pass
TC12	Get student by invalid ID	Random invalid ID	404 “Student not found”	404 returned	✅ Pass
TC13	Update student details	PUT with changed semester	200 Updated student	Updated correctly	✅ Pass
TC14	Delete student	Valid student ID	200 “Student deleted”	200 returned	✅ Pass
TC15	Student role accessing delete endpoint	Student JWT token	403 “not authorized”	403 returned	✅ Pass

---

Module: Fee Management

TC#	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC16	Process fee payment (Cash)	Student ID + amount + method=Cash	201 with receipt number	Receipt generated	✅ Pass
TC17	Process fee payment (UPI)	Student ID + method=UPI + ref	201 with receipt and ref	Correct	✅ Pass
TC18	Get payment history for student	Student ID	List of all payments	History returned	✅ Pass

TC#	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC19	Process payment with missing amount	No amount field	400 Validation error	400 returned	✅ Pass

---

Module: Payroll

TC#	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC20	Get payroll sheet	GET /api/payroll/sheet	All employee salary records	Sheet returned	✅ Pass
TC21	Process monthly payroll	Month = Feb 2026	Salary records created for all	Records created	✅ Pass
TC22	Get individual payslip	Teacher ID	Payslip with basic, deductions, net	Payslip correct	✅ Pass

---

Module: Security

TC#	Test Case Description	Expected Result	Actual Result	Status
TC23	Attempt NoSQL injection in login body	Input sanitized; no unauthorized access	Blocked	✅ Pass
TC24	Exceed rate limit (105 requests in 15 min)	429 Too Many Requests	429 returned after 100	✅ Pass
TC25	Password stored in plain text?	DB must show bcrypt hash	bcrypt hash confirmed in Atlas	✅ Pass
TC26	CORS — request from unauthorized domain	Blocked with CORS error	CORS rejected	✅ Pass

---

12.3 Test Results Summary

Module	Test Cases	Passed	Failed	Pass Rate
Authentication	6	6	0	100%
Student Management	9	9	0	100%
Fee Management	4	4	0	100%
Payroll	3	3	0	100%
Security	4	4	0	100%
TOTAL	**26**	**26**	**0**	**100%**

12.4 Bug Fixing Process

During the development and testing phases, the following issues were identified and resolved:

Bug #	Bug Description	Module	Fix Applied
BUG-01	CORS error when frontend called backend API	Auth	Added `cors({ origin: FRONTEND_URL })` in index.js
BUG-02	JWT token not persisted after page refresh	Frontend Auth	Switched Zustand store to `persist` middleware with localStorage
BUG-03	Password comparison always returning false	Auth	Fixed bcryptjs `hash` rounds — was using plain text comparison
BUG-04	Paginated student list returning incorrect total count	Student	Added `countDocuments(filter)` with the same filter as `find()`

Bug #	Bug Description	Module	Fix Applied
BUG-05	Fee receipt number had collision on fast requests	Finance	Changed from `Math.random()` to `Date.now()` + unique suffix
BUG-06	Mongoose `populate()` returning null for teacherProfileId	User	Fixed ObjectId reference — incorrect collection name in ref field
BUG-07	React Router protected route redirecting even when logged in	Frontend	Fixed Zustand `isAuthenticated` flag initialization from persisted state
BUG-08	Dashboard stats not updating after adding new student	Dashboard	Added `useEffect` dependency on navigation events

---

12.5 Final Deployment

Deployment Steps (Render.com)

```
 1. Push code to GitHub repository

2. Backend Deployment (Render.com Web Service):
   - Connect GitHub repository
   - Set Build Command: npm install
   - Set Start Command: node src/index.js
   - Add Environment Variables:
       MONGODB_URI = [MongoDB Atlas URI]
       JWT_SECRET  = [32-char random string]
       JWT_REFRESH_SECRET = [32-char random string]
       PORT = 5000
       NODE_ENV = production
       FRONTEND_URL = [frontend URL]

3. Frontend Deployment (Render.com Static Site):
   - Connect GitHub repository
   - Set Build Command: npm install && npm run build
```

```
    - Set Publish Directory: dist
    - Add Environment Variable:
        VITE_API_URL = [backend URL]/api

 4. MongoDB Atlas Setup:
    - Create free M0 cluster
    - Set IP Whitelist to 0.0.0.0/0 (allow all)
    - Create database user
    - Copy connection URI to backend env
```

Live URLs

Service	URL
Frontend	`https://college-management-frontend.onrender.com`
Backend API	`https://college-management-backend.onrender.com`
Database	MongoDB Atlas (Singapore region)

12.6 Conclusion

The **College Management System** was successfully developed, tested, and deployed. All 26 test cases passed with a 100% pass rate. The system fulfills all functional and non-functional requirements defined in the SRS. It provides a modern, secure, and scalable digital platform that effectively replaces manual paper-based college administration operations.

REFERENCES

#	Reference
1	MongoDB, Inc. *MongoDB Manual*. https://www.mongodb.com/docs/
2	OpenJS Foundation. *Node.js v18 Documentation*. https://nodejs.org/en/docs/
3	OpenJS Foundation. *Express.js 4.x API Reference*. https://expressjs.com/en/4x/api.html
4	Meta Open Source. *React 18 Official Documentation*. https://react.dev/
5	Mongoose Team. *Mongoose ODM Documentation v7*. https://mongoosejs.com/docs/
6	JWT. *JSON Web Tokens — Introduction*. https://jwt.io/introduction
7	Vite Team. *Vite Build Tool Documentation*. https://vitejs.dev/guide/
8	Tailwind Labs. *Tailwind CSS v3 Documentation*. https://tailwindcss.com/docs/
9	Render. *Render Deployment Documentation*. https://render.com/docs
10	Pressman, R.S. (2014). *Software Engineering: A Practitioner's Approach*, 8th Ed. McGraw-Hill
11	Tanenbaum, A.S. (2016). *Modern Operating Systems*, 4th Ed. Pearson
12	Silberschatz, A., Korth, H.F. & Sudarshan, S. (2020). *Database System Concepts*, 7th Ed. McGraw-Hill

© 2026 — College Management System | MERN Stack Department of Computer Science | [Institute Name] | Academic Year 2025–2026